

Prediction of Class A Vessel Locations from Automatic Information System Streamed Data

Group Assignment Report

Prepared by Group 11:

Craig Vande Ligt (vandeligt@gmail.com)

Colin Steensma (colinsteensma@gmail.com)

Rob Mifsud (robmifsud@outlook.com)

Collin Potter-Bonar (collin.potter.bonar@gmail.com)

Gabe Wolofsky (gabew@outlook.com)

Anca Rotaru (arotaru@waterloo.ca)

Table Of Contents

Introduction.....	1
Objective.....	1
Dataset Overview.....	1
Data Preparation.....	2
Data Streaming.....	2
Preprocessing & Feature Engineering.....	2
Model Design and Evaluation.....	3
Model Design.....	3
Model Evaluation and Selection.....	3
Model Selection and Selection Framework.....	5
Model Implementation.....	6
Model Deployment.....	6
Dynamic Visualization.....	6
Conclusion.....	7
Critique and Future Direction.....	8

Introduction

A key challenge in the area of big data management and analysis is working with data that has high velocity. This is particularly true in the case of streaming data. Streaming data is defined by its constantly changing nature since it is updated as events happen in real-time or near real-time. Examples of streaming data include stock prices, vehicle location, point-of-sale, etc. Related to the idea of reading in data in near real-time is the possibility of near real-time analysis of the data. Since the data is no longer static or of a fixed size, special approaches are needed for analysis steps related to identifying the source of streaming inputs, defining the data transformations, specifying the destination of outputs, defining the timing of the processing through triggering rules, and running the query to set the process in motion (and stop it!).

Objective

For our project, we chose to conduct a thorough analysis using a streaming data source tracking positions of marine vessels. We wanted to explore how each step of the data analysis process would be executed in this environment and identify challenges along the way. The steps we focused on were: data ingestion, data visualization, and development of a machine learning model to predict vessel movement.

Dataset Overview

The data source we chose to use is aisstream.io. The Automatic Information System (AIS) is a broadcast system that transmits the unique identification, position, course, and speed of a marine vessel. The International Maritime Organization's International Convention requires AIS to be fitted aboard international voyaging ships with 300+ gross tonnage and all passenger ships regardless of size. As such, this system captures a large subset of international marine shipping traffic. The site uses a network of

stations around the world to capture these messages and streams them to site users in JSON format. This data is open source and free to use once the user creates an API key. Different types of messages are available, but we will focus on the vessel position reports. Position reports are used to report the vessel's current position (longitude and latitude), heading, speed, and other relevant information, including the vessel's unique identifier.

Data Preparation

Data Streaming

As mentioned in the introduction, the first step of data preparation in a streaming context is to identify the source of streaming input. PySpark `DataStreamReader` can take a number of different source types, including traditional file formats (e.g., CSV, JSON, Parquet), Kafka, socket, and rate/rate per micro-batch. The first challenge we encountered with this data source is that its API uses a WebSocket protocol. While at first glance this seems to meet our requirement of “socket” source type included above, in reality, WebSocket is a more restricted type of protocol. WebSockets require a permanent connection to the server and are restricted to use of HTTP protocol. They are less flexible than sockets, which can be manually opened and closed.

Given the above limitation, our first attempt to ingest the data employed Kafka as a bridge between the AIS stream and PySpark. This would connect to the site via WebSocket, send a subscription that would filter for position report messages from vessels for a pre-defined period of time, send the messages in JSON format to our Kafka topic, parse the JSON, and extract the structured fields into a `DataFrame`. While making the initial connection was successful with this approach, we did encounter some challenges, including connecting to Kafka from Google Colab, maintaining connections, and gracefully stopping the ingestion process. For this reason, we moved on to an alternative approach for data ingestion similar to the example given in class, where we ingested a large number of position reports, packaged them in a parquet file, and then simulated the stream by reading in data in batches at a pre-defined rate.

The incoming streaming data via Kafka was run for a one-hour period. The aim was to acquire at least ten data points per Class A vessel to ensure that the data could be reduced at a later time while still providing enough data points per vessel for various vessel metrics over time. Upon completion of this stream, the dataset was compacted and downloaded as a parquet file.

Preprocessing & Feature Engineering

The compacted parquet file was ingested into Spark and filtered for all Class A vessels containing at least twenty data points. `Event_time` was created by parsing the static Spark `DataFrame`'s `time_utc` and applying a UTC formatting to each row. Data was then partitioned by a vessel (`mmsi`) window that was ordered by `event_ts`. A time delta column was created, indicating the delta in seconds between consecutive messages (`dt_prev_sec`), from which a rolling average of the prior ten data points (current data point included as the first value) were used to give the mean delta between messages (`dt_mean_sec`).

Lag features were also created based on the previous ten data points per vessel, and built upon latitude, longitude, Speed Over Ground (SOG), and Course Over Ground (COG). Targets were also generated using the next observed AIS data point per vessel.

Any rows that did not contain a full history (ten data points) or a next data point were dropped from the dataset, along with all NULL-containing rows.

To downsample the extensive dataset to a more manageable scale for the project's purpose, five hundred vessels were randomly selected, and their data rejoined to create a downsampled dataset. For ease of use in the notebook, the downsampled dataset was saved as a parquet file, with subsequent cells set up for its ingestion.

Model Design and Evaluation

Model Design

The downsampled dataset was ingested and split using an 80:20 random split to obtain a training (8943 rows) and testing dataset (2152 rows), respectively. An Assemble feature vector was then created using the lag, *dt_prev_sec*, and *dt_mean_sec* features. The output assembler training and testing datasets were cached to enhance the speed and efficiency of each machine learning model to be run. Three regression models (Random Forest, Gradient Boost Trees, and Linear/Elastic-Net) were designed, systematically trained, and fitted to the test dataset. Each model was trained on the latitudinal target using a hyperparameter-tuned cross-validation with three folds and an accuracy metric of Root Mean Square Error (RMSE) in degrees, and then transformed using the test dataset. Each model was then trained on the longitudinal target using the same methodology of training, but was then transformed on the latitudinally transformed test dataset, ensuring a sequential combination of both the latitudinal and longitudinal models. Several model evaluation metrics were then created on these sequential combination models, such as mean, median, and max error in km (kilometers), as well as RMSE in km.

Model Evaluation and Selection

The performance of the three regression models (Random Forest, Gradient Boosted Trees (GBT), and Linear/Elastic-Net) were assessed for geospatial coordinate prediction. The evaluation incorporated root mean squared error (RMSE), mean absolute error (MAE), mean absolute percentage error (MAPE), maximum error (Max), median error (Median), and the coefficient of determination (R-squared or R^2), calculated for latitude, longitude, and overall distance in kilometers. These metrics provide a multidimensional perspective on the accuracy and robustness of each model.

Table 1: Summary of Evaluation/Performance Metrics

Model	RMSE_lat	RMSE_lon	RMSE_km	MAE_lat	MAE_lon	MAE_km	Mean_km	Median_lat	Median_lon	Median_km	Max_lat	Max_lon	Max_km	MAPE_lat	MAPE_lon	R2_lat	R2_lon	R2_mean
Random Forest	0.4044	1.0613	113.5682	0.1282	0.2802	0.3529	32.5734	0.0006	-0.0	8.8646	7.5474	22.7674	2401.0845	0.0158	0.0428	0.9996	0.9997	0.9996
GBT	0.6391	1.9588	199.2891	0.2872	0.6281	0.7605	70.5922	-0.0052	-0.0017	27.6721	10.7873	47.842	4663.6149	0.0213	0.0738	0.999	0.9989	0.9989
Linear/ElasticNet	0.008	0.0121	1.2477	0.0034	0.0063	0.0079	0.6678	-0.0002	0.0002	0.3072	0.1371	0.183	19.0885	0.0001	0.0013	1.0	1.0	1.0

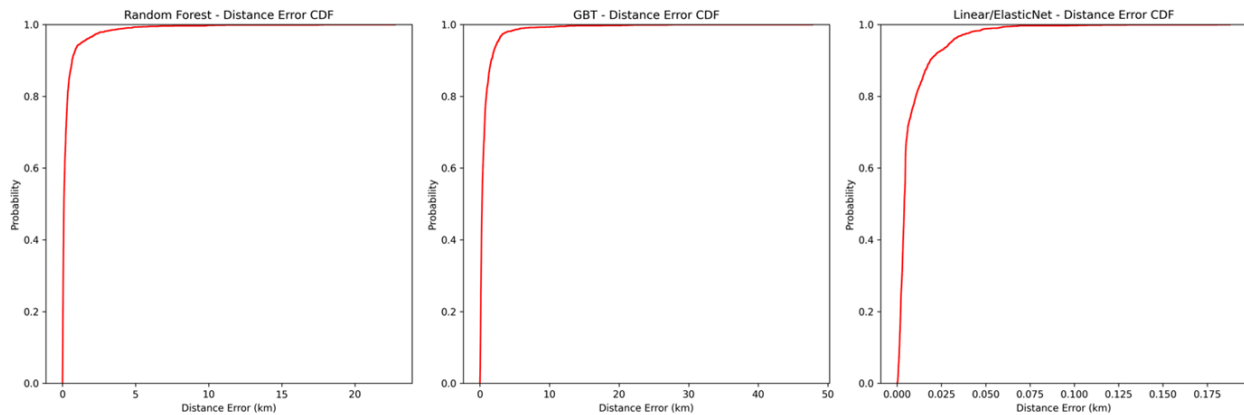
The Linear/Elastic-Net model consistently outperformed the other two approaches. It achieved exceptionally low RMSE values (0.0080 for latitude, 0.0121 for longitude, and 1.2477 km overall), indicating minimal deviation between predicted and observed coordinates. MAE values were similarly small (0.0034 for latitude, 0.0063 for longitude, and 0.0079 km), and the mean distance error was only 0.6678 km. Importantly, the median error remained well below one kilometer (0.3072 km), showing that the model's typical prediction error was modest. Even in the worst case, the maximum error was limited to 0.1830 km, substantially lower than those observed for Random Forest and GBT. Furthermore, the Linear/Elastic-Net model produced near-zero MAPE values (0.0001 for latitude and 0.0013 for longitude) and perfect R^2 scores (1.0000 for both latitude and longitude), demonstrating its ability to explain virtually all variance in the data.

By contrast, the Random Forest model performed moderately well, with RMSE_km of 113.5682 and MAE_km of 0.3529. Although its R^2 values were high (0.9996 for latitude and 0.9997 for longitude), the model produced substantially larger mean errors (32.5734 km) and a median error of 8.8646 km. Its maximum error reached 22.7674 km, highlighting occasional large deviations from the true values. MAPE values (0.0158 for latitude and 0.0428 for longitude) were also higher than those of Linear/Elastic-Net, reflecting weaker relative accuracy.

The GBT model performed the weakest overall, with RMSE_km of 199.2891 and MAE_km of 0.7605. Its mean error was 70.5922 km, and the median error rose to 27.6721 km, indicating that typical predictions were far less reliable. The maximum error reached 47.8420 km, the largest among the three models, underscoring its instability. MAPE values (0.0213 for latitude and 0.0738 for longitude) were the highest overall, and although R^2 values remained strong (0.9990 and 0.9989), they did not compensate for the poor error distribution.

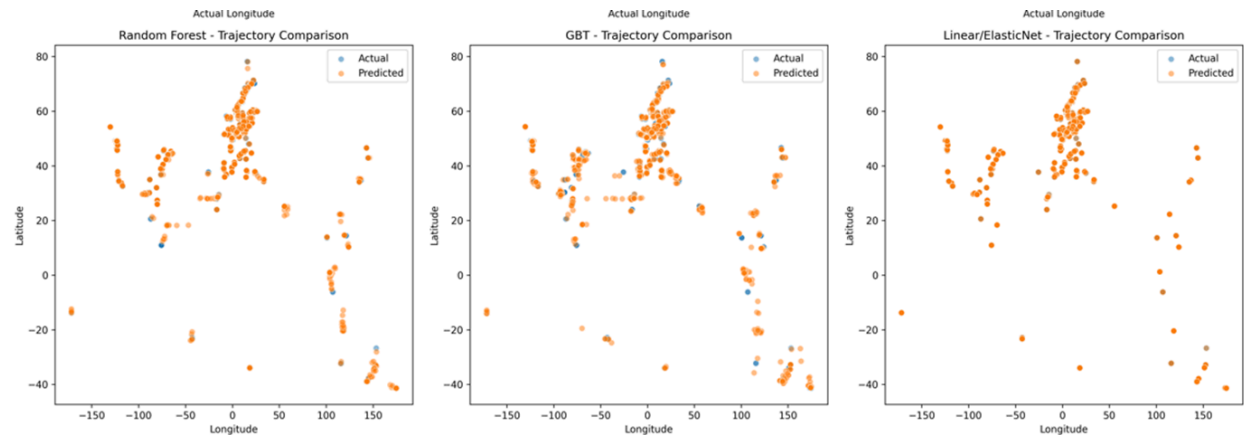
The Cumulative Distribution Function (CDF) curves generated for each model provide additional insight into the distribution of errors. For the Linear/Elastic-Net model, the latitude, longitude, and distance error curves rise sharply near zero, indicating that the vast majority of predictions fall within extremely small error margins. By comparison, the Random Forest and GBT curves rise more gradually, reflecting broader error distributions and a higher proportion of large deviations. The steepness of the Linear/Elastic-Net CDF curves confirms its superior consistency and reliability, as errors are tightly concentrated around zero rather than spread across a wide range.

Figure 1: CDF curves for each model expressed in kilometers



The trajectory comparison plots visually reinforce these findings. The Linear/Elastic-Net model shows near-perfect overlap between actual and predicted trajectories, remaining closely aligned with the actual data. In contrast, the Random Forest and GBT plots reveal noticeable dispersion, particularly in longitudinal predictions, where the predicted paths deviate significantly from the actual trajectories. These visual discrepancies align with the higher error metrics observed in the tabular analysis.

Figure 2: Scatterplots illustrating the actual and projected trajectories of vessels



Model Selection and Selection Framework

To formalize the model selection process, a function named `[select_best_model_from_summary]` was developed. This function evaluates a summary DataFrame, `df_evaluation_summary`, which consolidates all computed performance metrics for the potential models. By systematically assessing these metrics, the function determines and returns the model that demonstrates the most optimal performance.

The function prioritizes `RMSE_km` as the primary selection criterion and employs the average R-squared of longitude and latitude (`R2_mean`) as a secondary criterion, a criterion intended to serve as a ‘tie-breaker’. In the context of this research, `R2_mean` is defined as the average of `R2_lat` and `R2_lon`. It provides a single measure of overall explanatory power across latitude and longitude. The function then

sorts the evaluation summary by RMSE_km and R2_mean, selecting the model with the lowest error and highest explanatory strength. In this research, the function identified the Linear/Elastic-Net model as the best performer, consistent with both the quantitative metrics and the visual diagnostics.

Model Implementation

Model Deployment

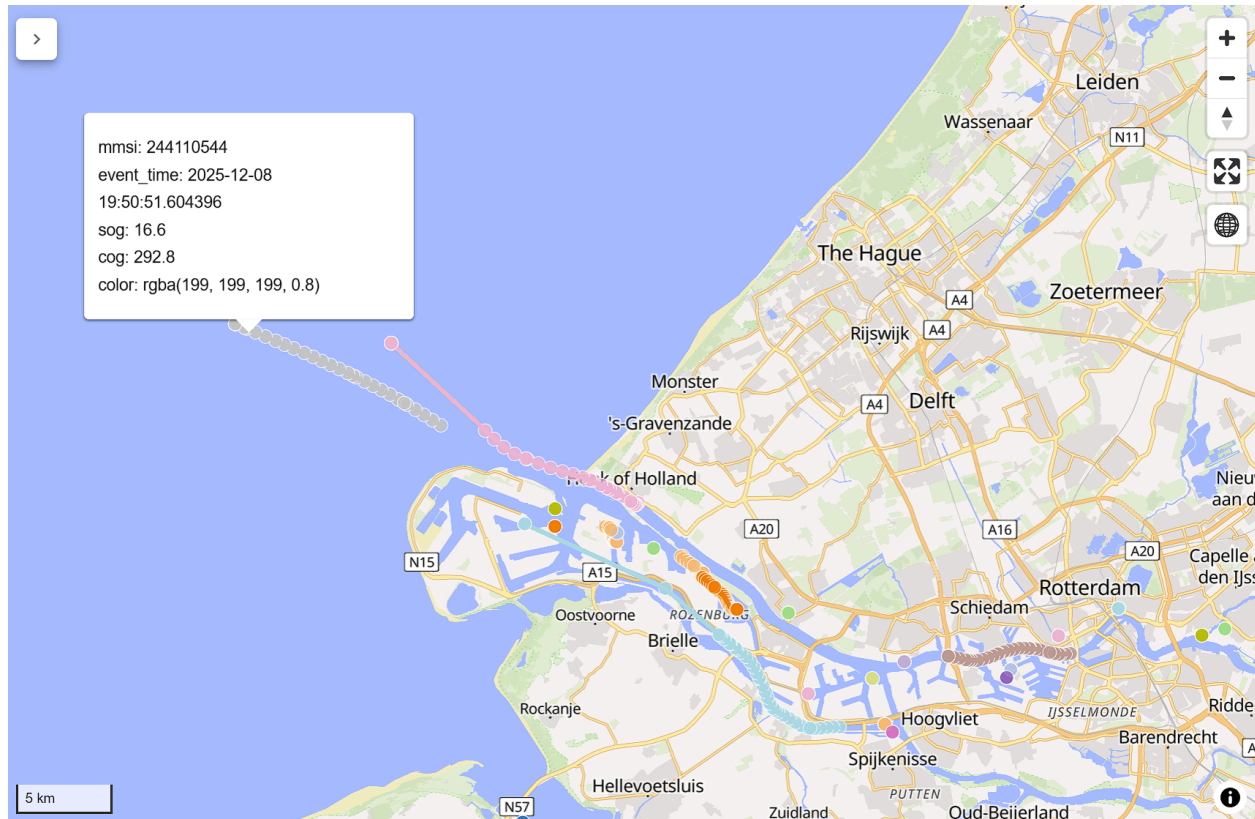
Due to time constraints and the complexity of using lag features with streamed data (requiring historic data of at least 9 data points per vessel for inference and mapping), a stream-simulated architecture was utilized in which static historical data was treated as a stream of incoming events. Using this approach, the incoming data was cleaned, features were engineered, and locations were inferred for both the latitude and longitude.

Dynamic Visualization

An interactive and dynamic Python Leafmap was made to visualize a sample of the streamed data as a proof of concept for a visualization supporting real-time data. The open source leafmap.maplibregl package was chosen for its high performance at rendering large datasets in Jupyter Notebooks.

The sample parquet file of the ship's positions was used for the visualization aspect to reduce complexity of the task and enable faster testing without waiting for the stream to accrue data. Ship positions' timestamps were ordered and normalized in order to display a new position for each ship every second when the map updates, instead of the sometimes infrequent refresh rate in the raw data stream. Each ship was assigned a unique colour by the *mmsi* number. A starting viewpoint around Rotterdam was established as this location provided a multitude of ship movements. The user is free to scroll around the map to other regions - the zoom level stays the same upon layer refresh, which was accomplished using the `fit_bounds=False` option.

Figure 3: Screenshot of Vessel Position Visualization



Future visualization work would include displaying the live positions and history from the AIS stream, and to incorporate the machine learning prediction model in order to show the ship's predicted locations.

Conclusion

The project successfully demonstrated the end-to-end process of ingesting, preparing, visualizing, and modeling high-velocity streaming data from the Automatic Information System (AIS). We navigated the initial challenges of integrating a WebSocket-based data source by transitioning to a simulated stream from a processed Parquet file during the deployment stage.

The core achievement was the development and evaluation of a machine learning model to predict the future geospatial coordinates of Class A marine vessels. Through comparative analysis, the Linear/Elastic-Net regression model was selected as the optimal choice. This model exhibited superior performance, achieving an RMSE of 1.2477 km, a mean distance error of only 0.6678 km, and an exceptionally low median error of 0.3072 km, alongside perfect R^2 scores. The linear nature of the model effectively captured the fundamental physics of vessel movement—ships generally moving at a constant velocity vector—allowing it to extrapolate position accurately. In contrast, tree-based regression models, like Random Forest and Gradient Boosted Trees (GBT), performed poorly for coordinate prediction due to their inherent inability to extrapolate values beyond the range of the training data, leading to large, unpredictable errors when a vessel moved into an unseen geographical area.

Critique and Future Direction

While the Linear/Elastic-Net model proved robust for predicting vessel positions during straight-line travel, its primary limitation is the assumption of perpetual straight-line movement, which causes prediction error during complex maneuvers such as turning or docking.

To enhance the project's real-world applicability and accuracy, several critical next steps are recommended:

- **Model Enhancement** (Target Transformation): The most immediate improvement would involve changing the model's prediction target. Instead of predicting the absolute future coordinates (Latitude/Longitude), the model should predict the change in coordinates (Delta Latitude/Delta Longitude). This transformation standardizes the target variable, allowing tree-based models like Random Forest to be effective, as the physics of movement (the magnitude of the delta) remains consistent regardless of the vessel's current global location.
- **Adopting Advanced Time-Series Models**: For the highest accuracy, especially when modeling complex, non-linear trajectories, the implementation of Recurrent Neural Networks (RNNs) or Long Short-Term Memory (LSTM) networks is warranted. These models intrinsically handle sequential data and internalize vessel history without the need for manual lag feature engineering.

Moving to Near Real-Time Infrastructure: To transition the prototype to a production-ready system, the following infrastructure changes are necessary:

- Replacing the simulated Parquet stream with a live Kafka topic to handle the AIS vessel data in near real-time.
- Implementing Spark Stateful Streaming (using `flatMapGroupsWithState`) to maintain a rolling history of each vessel's track in memory, eliminating reliance on disk-based storage and enabling sub-second prediction latency.

In conclusion, the project successfully established a foundational framework for streaming AIS vessel data, developing and deploying a machine learning model, and demonstrated the efficacy of simple linear models for baseline trajectory prediction. The path forward involves refining the machine learning approach with more advanced techniques and hardening the data pipeline for true, low-latency, near real-time operation.

References

AIS Stream. (2022). *Track the world's vessels via WebSocket*. Retrieved December 15, 2025, from <https://aisstream.io>